LARAVEL

Request Handling & Form Processing

GET and POST • CSRF Protection • Old Input • Flash Data • Validation

Learning outcome By the end of this guide, students should be able to build a secure Laravel form, receive and inspect submitted data, validate it on the server, return useful feedback, and preserve user input after validation errors.

Complete teaching notes with explanations, code, examples, classroom workflow and practice tasks

Contents

- 1. Learning objectives and prerequisites
- 2. How an HTTP form request works
- 3. GET and POST methods
- 4. Routes and controller request handling
- 5. The Laravel Request object
- 6. Building a complete form
- 7. CSRF protection
- 8. Redirects, old input and flash data
- 9. Validation workflow
- 10. Built-in validation rules
- 11. Custom validation messages and attributes
- 12. Displaying validation errors in Blade
- 13. Complete practical example: student registration
- 14. File uploads and validation
- 15. Testing with browser and Thunder Client
- 16. Common errors and troubleshooting
- 17. Security and best practices
- 18. Classroom exercises and assessment questions
- 19. Quick revision sheet

1. Learning objectives and prerequisites

This guide uses Laravel 12-style syntax and Blade templates. The same core request, routing, CSRF and validation concepts apply across recent Laravel versions. Confirm version-specific details against the project installed in your classroom.

Prerequisites:

- Basic PHP syntax, arrays and functions
- HTML forms and input types
- Basic MVC understanding
- Laravel project setup, routes, controllers and Blade views
- Basic database and Eloquent knowledge (helpful for the database validation examples)

After completing the lesson, students should be able to:

- Explain the difference between GET and POST.
- Create routes that receive form submissions.
- Read submitted values using the Request object.
- Protect browser forms with CSRF tokens.
- Redirect users and preserve old input using flash data.
- Validate data with built-in rules and custom messages.
- Show field-level and summary validation feedback.
- Explain why server-side validation is essential even when HTML validation exists.

2. How an HTTP form request works

A form is an HTML interface that collects information. When the user clicks Submit, the browser sends an HTTP request to a URL. Laravel matches that URL and HTTP method to a route, runs the controller or route callback, reads the request data, validates it, and then returns a response.

```
Browser / HTML form
        |
        |  HTTP request (GET or POST)
        v
Laravel Route
        |
        v
Controller action
        |
        +--> Read input from Request
        |
        +--> Validate input
        |
        +--> Perform action (save / update / search)
        |
        v
Response: View / Redirect / JSON
```

Important terms:

- Request: data and metadata sent by the browser, such as form fields, query parameters, headers and uploaded files.
- Route: maps an HTTP method and URL to application code.
- Controller: contains the application logic for handling the request.
- Response: what Laravel sends back, such as HTML, JSON or a redirect.

- Validation: checking whether submitted data satisfies the application's rules.

3. GET and POST methods

3.1 GET method

GET is generally used to retrieve or display information. Search forms and filter forms commonly use GET. Submitted values usually appear in the URL as query parameters.

```
<!-- resources/views/products/search.blade.php -->
<form action="{{ route('products.search') }}" method="GET">
    <label for="keyword">Search product</label>
    <input type="text" id="keyword" name="keyword">
    <button type="submit">Search</button>
</form>
```

```
// routes/web.php
use Illuminate\Support\Facades\Route;
use Illuminate\Http\Request;

Route::get('/products/search', function (Request $request) {
    $keyword = $request->query('keyword');

    return 'Searching for: ' . e($keyword);
})->name('products.search');
```

Example URL after submission: /products/search?keyword=laptop

GET characteristics:

- Designed for retrieval, not for changing server data.
- Parameters are visible in the URL and may be saved in browser history or shared.
- Can be bookmarked and cached depending on the response and cache rules.
- Do not put passwords, OTPs, private tokens or sensitive personal information in a GET query string.

3.2 POST method

POST is commonly used to submit data that creates or changes something, such as registering a student, saving a profile or creating an order. The form data is sent in the request body rather than being appended to the URL.

```
<!-- resources/views/students/create.blade.php -->
<form action="{{ route('students.store') }}" method="POST">
    @csrf

    <label>Name</label>
    <input type="text" name="name">

    <label>Email</label>
    <input type="email" name="email">

    <button type="submit">Save Student</button>
</form>
```

```
// routes/web.php
use App\Http\Controllers\StudentController;

Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');

Route::post('/students', [StudentController::class, 'store'])
    ->name('students.store');
```

GET versus POST:

Typical purpose	Retrieve/search/filter	Submit/create/change data
Data location	URL query string	Request body
Bookmark/share	Often suitable	Not normally meaningful
Sensitive data	Avoid	Body is less exposed in URL, but HTTPS is still mandatory
Laravel route	Route::get(...)	Route::post(...)

Important correction POST does not automatically make data secure or encrypted. HTTPS provides transport encryption. CSRF protection and authorization solve different security problems.

4. Routes and controller request handling

For a clean MVC structure, keep route definitions in routes/web.php and put request-handling logic in a controller.

Create a controller:

```
php artisan make:controller StudentController
```

Example controller:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    // Display the form
    public function create()
    {
        return view('students.create');
    }

    // Receive and process submitted form data
    public function store(Request $request)
    {
        $name = $request->input('name');
        $email = $request->input('email');

        return "Name: {$name}, Email: {$email}";
    }
}
```

Routes:

```
use App\Http\Controllers\StudentController;

Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');

Route::post('/students', [StudentController::class, 'store'])
    ->name('students.store');
```

The type-hinted Request parameter is injected by Laravel's service container. Laravel supplies the current HTTP request object when the route calls the controller method.

5. The Laravel Request object

The Request object gives access to submitted values and request information. Import it with use Illuminate\Http\Request; in a controller.

5.1 Reading input

```
// Get one input value
$name = $request->input('name');

// Get a value with a default if it is absent
$city = $request->input('city', 'Durgapur');

// Read query-string data specifically
$keyword = $request->query('keyword');

// Read all input data
$allData = $request->all();

// Read only selected keys
$data = $request->only(['name', 'email']);

// Exclude selected keys
$data = $request->except(['password', 'password_confirmation']);

// Check whether a key is present
if ($request->has('email')) {
    // key exists in the request
}

// Check whether a key is present and not empty
if ($request->filled('name')) {
    // name has a non-empty value
}

// Check whether a checkbox/key was sent
if ($request->boolean('is_active')) {
    // interprets common truthy values as boolean
}
```

input() versus query():

- input('keyword') reads from the request input payload and can also account for query input.
- query('keyword') reads specifically from the URL query string.

Do not confuse these two expressions:

```
$request->input('name');  // Correct: calls the input() method
$request->input['name'];  // Incorrect: input is a method, not an array property
```

5.2 Checking request method and path

```
$method = $request->method();       // GET, POST, PUT, PATCH, DELETE
$isPost = $request->isMethod('POST');
$path = $request->path();              // e.g. students
$url = $request->url();                // URL without query string
$fullUrl = $request->fullUrl();        // URL including query string
```

5.3 Safe data handling

Never assume that browser-submitted data is correct. A user can modify HTML, disable JavaScript, or send a request using a tool such as curl or Thunder Client. Validate the input and pass only approved fields to the database.

```
// Prefer validated data
$validated = $request->validate([
    'name' => ['required', 'string', 'max:100'],
    'email' => ['required', 'email'],
]);

// Use only the validated fields for persistence
// Student::create($validated);
```

6. Building a complete form

A Blade form should have a destination route, an HTTP method, named fields and a submit button. The name attribute is essential because Laravel uses it as the input key.

```
<!-- resources/views/students/create.blade.php -->
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
    <h1>Student Registration</h1>

    <form action="{{ route('students.store') }}" method="POST">
        @csrf

        <div>
            <label for="name">Full name</label>
            <input type="text" id="name" name="name">
        </div>

        <div>
            <label for="email">Email</label>
            <input type="email" id="email" name="email">
        </div>

        <div>
            <label for="phone">Phone</label>
            <input type="text" id="phone" name="phone">
        </div>

        <button type="submit">Register</button>
    </form>
</body>
</html>
```

Teaching point The id attribute connects a label to an HTML element. The name attribute determines the key sent to the server. An input without a name is generally not submitted as a form field.

7. CSRF protection

7.1 What is CSRF?

CSRF means Cross-Site Request Forgery. It is an attack in which a malicious website attempts to make a user's browser send an unwanted request to another website where the user is already authenticated.

Example scenario:

1. A user signs in to an online banking website.
2. The browser keeps the bank's authentication cookie.
3. The user visits a malicious page in another tab.
4. That page tries to submit a hidden request to the bank.
5. Without suitable protection, the browser may attach the bank cookie to the request.

A CSRF token helps Laravel verify that a state-changing browser request came through a form or session associated with the application.

7.2 Adding the CSRF token

```
<form action="{{ route('students.store') }}" method="POST">
    @csrf

    <input type="text" name="name">
    <button type="submit">Save</button>
</form>
```

The @csrf Blade directive generates a hidden input similar to:

```
<input type="hidden" name="_token" value="generated-session-token">
```

The token is generated dynamically. Do not manually copy a token from one request and hard-code it into a form.

7.3 What happens if @csrf is missing?

For routes using Laravel's web middleware, a POST/PUT/PATCH/DELETE request without a valid CSRF token is normally rejected. In a typical browser request, the user may see HTTP 419 (Page Expired).

7.4 CSRF in GET and POST

- GET requests should be read-only and should not perform actions such as deleting records or changing account settings.
- POST, PUT, PATCH and DELETE browser form requests need CSRF protection when handled through the web middleware.
- Do not change a destructive route to GET just to avoid CSRF errors. Fix the form and route correctly.

7.5 CSRF and APIs

Laravel API routes are generally stateless and do not use the same session-based CSRF flow as browser routes in routes/web.php. API authentication commonly uses bearer tokens or another API authentication mechanism. If an

API uses cookie-based browser authentication, its CSRF requirements must be configured correctly; do not assume every API is automatically safe.

7.6 CSRF versus authentication and authorization

Authentication	Who are you?	Confirms user identity
Authorization	Are you allowed to do this?	Checks permissions
CSRF protection	Was this browser action intentionally initiated from the application?	Helps prevent forged cross-site requests

8. Redirects, old input and flash data

8.1 Redirect response

A redirect tells the browser to make a new request to another URL. It is commonly used after a form is processed.

```
return redirect()->route('students.create');
```

After a successful POST, redirecting instead of returning the form view directly helps prevent accidental resubmission when the user refreshes the page. This is commonly called the Post/Redirect/Get pattern.

8.2 Flash data

Flash data is session data intended to be available during the next request. It is useful for one-time messages such as success, warning or error notifications.

```
// Controller
return redirect()
    ->route('students.create')
    ->with('success', 'Student registered successfully.');

<!-- Blade view -->
@if (session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
    </div>
@endif
```

Other examples:

```
return back()->with('warning', 'Please review the submitted details.');
return back()->with('error', 'Unable to complete the operation.');
```

8.3 Old input

When validation fails, Laravel automatically redirects back and flashes the submitted input to the session, except sensitive fields such as passwords are not flashed by default. Blade's old() helper retrieves a previously submitted value.

```
<input type="text" name="name" value="{{ old('name') }}">
<input type="email" name="email" value="{{ old('email') }}">
```

For a select box:

```
<select name="city">
    <option value="">Choose city</option>
    <option value="Durgapur" {{ old('city') == 'Durgapur' ? 'selected' : '' }}>
        Durgapur
    </option>
    <option value="Kolkata" {{ old('city') == 'Kolkata' ? 'selected' : '' }}>
        Kolkata
    </option>
</select>
```

For a checkbox:

```
<input type="checkbox" name="terms" value="1"
    {{ old('terms') ? 'checked' : '' }}>
<label>I accept the terms</label>
```

8.4 Old input versus flash data

What it stores	Previously submitted form values	Temporary session values
Common use	Refill form fields after an error	Show one-time success/error message
Blade access	old('field')	session('key')
Typical lifecycle	Available after redirect for the next request	Available for the next request, then expires

9. Validation workflow

Validation is the server-side process of checking submitted data before using it. HTML attributes such as required, minlength and type=email improve the user experience, but they are not a security boundary.

9.1 Typical validation sequence

6. User opens the form using a GET route.
7. User enters values and submits the form using POST.
8. Laravel receives the request.
9. The controller applies validation rules.
10. If validation passes, Laravel returns validated data to the controller.
11. If validation fails, Laravel redirects back, stores errors in the session and flashes eligible input.
12. The Blade view displays errors and restores old input.

9.2 Quick validation using validate()

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'max:255'],
        'age' => ['required', 'integer', 'min:18', 'max:60'],
    ]);

    // Use only validated values
    // Student::create($validated);

    return redirect()
        ->route('students.create')
        ->with('success', 'Student registered successfully.');
}
```

If the rules pass, validate() returns an array containing validated data. If they fail in a traditional web request, Laravel automatically redirects back with validation errors and eligible old input. For JSON requests, Laravel normally returns a 422 Unprocessable Entity response with validation details.

9.3 Manual Validator

Use the Validator facade when you need more control over the validation flow.

```
use Illuminate\Support\Facades\Validator;

public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'name' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email'],
    ]);

    if ($validator->fails()) {
        return redirect()
            ->back()
            ->withErrors($validator)
            ->withInput();
    }

    $validated = $validator->validated();

    return redirect()
        ->route('students.create')
        ->with('success', 'Data is valid.');
}
```

Use validate() for the common case. Use Validator::make() when you need to inspect errors, add conditional rules, or control the response yourself.

9.4 Validated data is not the same as all request data

```
$all = $request->all();             // Every submitted input
$validated = $request->validate([...]); // Only data that passed the rules
```

Best practice Do not pass $request->all() directly into a model's create() or update() method. Validate first, then persist only the validated fields. Also configure model fillable/guarded properties appropriately.

10. Built-in validation rules

Rules may be written as a pipe-separated string or as an array. The array form is often easier to read, especially when rules contain parameters.

```
// String syntax
'name' => 'required|string|max:100',

// Array syntax
'name' => ['required', 'string', 'max:100'],
```

required	Field must be present and not empty.
nullable	Null is allowed; other rules are generally skipped for a null value.
sometimes	Apply rules only if the field is present in the input.
string	Value must be a string.
integer	Value must be an integer.
numeric	Value must be numeric.
boolean	Value must be a boolean-like value accepted by Laravel.
email	Value must be a valid email address.
url	Value must be a valid URL.
alpha	Only alphabetic characters.
alpha_num	Only letters and numbers.
alpha_dash	Letters, numbers, dashes and underscores.
min:5	Minimum size/value is 5; interpretation depends on the data type.
max:100	Maximum size/value is 100; interpretation depends on the data type.
between:1,10	Value must be between 1 and 10.
digits:10	Must contain exactly 10 digits.
digits_between:10,12	Must contain between 10 and 12 digits.
size:10	Must have exactly the specified size/value.
in:active,inactive	Value must be one of the listed values.
not_in:admin,root	Value must not be one of the listed values.
confirmed	Requires a matching field named field_confirmation.
same:password	Must match another field.
different:username	Must differ from another field.
date	Must be a valid date.
date_format:Y-m-d	Must match the specified date format.
before:today	Must be before the given date.
after:today	Must be after the given date.
accepted	Must be accepted, commonly used for terms and conditions.
required_if:role,admin	Required when another field equals the specified value.
required_with:phone	Required when the other field is present.

required_without:email	Required when the other field is missing.
unique:users,email	Value must be unique in the specified table/column.
exists:departments,id	Value must exist in the specified table/column.
regex:/pattern/	Must match a regular expression.
image	Uploaded file must be an image.
mimes:jpg,png,pdf	Uploaded file must have one of the listed extensions/types.
mimetypes:image/jpeg,image/png	Uploaded file must have one of the listed MIME types.
file	Must be a successfully uploaded file.
max:2048	For a file, maximum size is 2048 KB.

Note: Some rules behave differently depending on the field type. For example, min and max can refer to string length, numeric value, array size or file size. Check the current Laravel validation documentation for edge cases and rule parameters.

10.1 Common combinations

```
$validated = $request->validate([
    'name' => ['required', 'string', 'min:2', 'max:100'],
    'email' => ['required', 'email', 'max:255', 'unique:users,email'],
    'age' => ['required', 'integer', 'between:18,60'],
    'phone' => ['required', 'digits:10'],
    'password' => ['required', 'string', 'min:8', 'confirmed'],
    'terms' => ['accepted'],
]);
```

For the confirmed rule, the form must include password_confirmation:

```
<input type="password" name="password">
<input type="password" name="password_confirmation">
```

10.2 Unique and exists rules

```
// Email must not already exist in users.email
'email' => ['required', 'email', 'unique:users,email'],

// Department ID must exist in departments.id
'department_id' => ['required', 'exists:departments,id'],
```

When updating an existing record, a unique rule usually needs to ignore the current record. Prefer the fluent Rule API and pass a trusted model/record identifier rather than trusting an arbitrary request value.

```
use Illuminate\Validation\Rule;

// Example for updating a user record
'email' => [
    'required',
    'email',
    Rule::unique('users', 'email')->ignore($user->id),
],
```

11. Custom validation messages and attributes

11.1 Custom messages in validate()

```
$validated = $request->validate(
    [
        'name' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email'],
        'age' => ['required', 'integer', 'min:18'],
    ],
    [
        'name.required' => 'Please enter your full name.',
        'name.max' => 'Your name cannot exceed 100 characters.',
        'email.required' => 'Email address is required.',
        'email.email' => 'Enter a valid email address.',
        'age.min' => 'You must be at least 18 years old.',
    ]
);
```

The message key uses field.rule format. For example, email.required customizes the required rule for the email field.

11.2 Custom attribute names

```
$validated = $request->validate(
    [
        'dob' => ['required', 'date'],
    ],
    [],
    [
        'dob' => 'date of birth',
    ]
);
```

Laravel can now use a human-friendly label such as “date of birth” in generated validation messages.

11.3 Custom messages in a Form Request

For larger forms, move validation into a dedicated Form Request class.

```
php artisan make:request StoreStudentRequest
```

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreStudentRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // Add a real permission check in protected applications
    }

    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:100'],
            'email' => ['required', 'email', 'max:255'],
            'phone' => ['required', 'digits:10'],
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'Please enter the student name.',
            'email.email' => 'Please enter a valid email address.',
            'phone.digits' => 'Phone number must contain exactly 10 digits.',
        ];
    }

    public function attributes(): array
    {
        return [
            'phone' => 'mobile number',
        ];
    }
}
```

Inject the Form Request into the controller. Laravel validates it before the controller action runs.

```php
use App\Http\Requests\StoreStudentRequest;

public function store(StoreStudentRequest $request)
{
    $validated = $request->validated();

    // Student::create($validated);

    return redirect()
        ->route('students.create')
        ->with('success', 'Student saved.');
}
```

Important authorize() returning true is acceptable for a simple classroom demonstration. In a real application, implement authorization so users cannot submit records they are not permitted to create.

12. Displaying validation errors in Blade

12.1 Show one field's error

```
@error('name')
    <div class="text-danger">{{ $message }}</div>
@enderror
```

12.2 Show all errors

```
@if ($errors->any())
    <div class="alert alert-danger">
        <strong>Please correct the following errors:</strong>
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
```

12.3 Restore old values and show field errors

```
<div>
    <label for="name">Name</label>
    <input
        type="text"
        id="name"
        name="name"
        value="{{ old('name') }}"
        class="@error('name') is-invalid @enderror"
    >

    @error('name')
        <small class="text-danger">{{ $message }}</small>
    @enderror
</div>

<div>
    <label for="email">Email</label>
    <input
        type="email"
        id="email"
        name="email"
        value="{{ old('email') }}"
        class="@error('email') is-invalid @enderror"
    >

    @error('email')
        <small class="text-danger">{{ $message }}</small>
    @enderror
</div>
```

Use Blade's escaped output syntax {{ }} for user-provided values and error messages. Avoid rendering untrusted values with {!! !!} unless you have a specific, safe reason.

13. Complete practical example: student registration

This example combines routes, controller, Blade form, CSRF, validation, custom messages, old input, error feedback and a success flash message.

Step 1 — Create the controller

```
php artisan make:controller StudentController
```

Step 2 — Add routes

```
// routes/web.php
use App\Http\Controllers\StudentController;

Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');

Route::post('/students', [StudentController::class, 'store'])
    ->name('students.store');
```

Step 3 — Controller code

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function create()
    {
        return view('students.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate(
            [
                'name' => ['required', 'string', 'min:2', 'max:100'],
                'email' => ['required', 'email', 'max:255'],
                'phone' => ['required', 'digits:10'],
                'age' => ['required', 'integer', 'between:16,60'],
                'course' => ['required', 'in:PHP,Laravel,Python,JavaScript'],
                'terms' => ['accepted'],
            ],
            [
                'name.required' => 'Student name is required.',
                'name.min' => 'Name must contain at least 2 characters.',
                'email.required' => 'Please enter an email address.',
                'email.email' => 'Please enter a valid email address.',
                'phone.digits' => 'Phone number must contain exactly 10 digits.',
                'age.between' => 'Age must be between 16 and 60.',
                'course.in' => 'Please select a valid course.',
                'terms.accepted' => 'You must accept the terms and conditions.',
            ]
        );

        // Demonstration only:
        // In a database-backed application, save $validated using a model.
        // Example: Student::create($validated);

        return redirect()
            ->route('students.create')
            ->with('success', 'Student registration completed successfully.');
    }
}
```

Step 4 — Create the Blade view

Create resources/views/students/create.blade.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Registration</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 650px;
               margin: 30px auto; padding: 0 16px; }
        .field { margin-bottom: 14px; }
        label { display: block; margin-bottom: 5px; }
        input, select { width: 100%; padding: 9px; box-sizing: border-box; }
        .error { color: #b00020; font-size: 13px; }
        .success { background: #e6ffed; padding: 10px; }
        .summary { background: #ffe8e8; padding: 10px; }
        button { padding: 10px 18px; cursor: pointer; }
    </style>
</head>
<body>
    <h1>Student Registration</h1>

    @if (session('success'))
        <div class="success">{{ session('success') }}</div>
    @endif

    @if ($errors->any())
        <div class="summary">
            <strong>Please fix these errors:</strong>
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form action="{{ route('students.store') }}" method="POST">
        @csrf

        <div class="field">
            <label for="name">Student name</label>
            <input id="name" name="name" type="text"
                   value="{{ old('name') }}">
            @error('name') <div class="error">{{ $message }}</div> @enderror
        </div>

        <div class="field">
            <label for="email">Email</label>
            <input id="email" name="email" type="email"
                   value="{{ old('email') }}">
            @error('email') <div class="error">{{ $message }}</div> @enderror
        </div>

        <div class="field">
            <label for="phone">Phone number</label>
            <input id="phone" name="phone" type="text"
                   value="{{ old('phone') }}">
            @error('phone') <div class="error">{{ $message }}</div> @enderror
        </div>

        <div class="field">
```

```
            <label for="age">Age</label>
            <input id="age" name="age" type="number"
                   value="{{ old('age') }}">
            @error('age') <div class="error">{{ $message }}</div> @enderror
        </div>

        <div class="field">
            <label for="course">Course</label>
            <select id="course" name="course">
                <option value="">Select a course</option>
                @foreach (['PHP', 'Laravel', 'Python', 'JavaScript'] as $course)
                    <option value="{{ $course }}"
                        {{ old('course') == $course ? 'selected' : '' }}>
                        {{ $course }}
                    </option>
                @endforeach
            </select>
            @error('course') <div class="error">{{ $message }}</div> @enderror
        </div>

        <div class="field">
            <label>
                <input type="checkbox" name="terms" value="1"
                       {{ old('terms') ? 'checked' : '' }}>
                I accept the terms and conditions
            </label>
            @error('terms') <div class="error">{{ $message }}</div> @enderror
        </div>

        <button type="submit">Register Student</button>
    </form>
</body>
</html>
```

Step 5 — Run and test

```
php artisan serve
```

Open http://127.0.0.1:8000/students/create in the browser.

Test cases:

13. Submit the form empty. Required-field errors should appear.
14. Enter an invalid email. The email error should appear.
15. Enter a phone number with fewer or more than 10 digits.
16. Enter age below 16 or above 60.
17. Do not accept terms and conditions.
18. Submit valid values. The success flash message should appear.
19. After a validation error, check that previously entered values remain in the form.

14. File uploads and validation

To upload a file, the form must use enctype="multipart/form-data". Read uploaded files through the Request object's file methods and validate them before storing.

```
<form action="{{ route('documents.store') }}"
      method="POST" enctype="multipart/form-data">
    @csrf

    <label>Document</label>
    <input type="file" name="document">

    @error('document')
        <div>{{ $message }}</div>
    @enderror

    <button type="submit">Upload</button>
</form>
```

```
public function store(Request $request)
{
    $validated = $request->validate([
        'document' => ['required', 'file', 'mimes:pdf,doc,docx', 'max:5120'],
    ]);

    $path = $request->file('document')->store('documents', 'public');

    return back()->with('success', 'File uploaded successfully.');
}
```

This example allows PDF, DOC and DOCX files up to 5120 KB (5 MB). Configure the public storage link if files must be served from the public disk:

```
php artisan storage:link
```

Security note Do not trust the original filename or file extension alone. Validate uploads, use controlled storage locations, limit file size, and avoid executing uploaded files as code.

15. Testing with browser and Thunder Client

15.1 Browser testing

20. Start the Laravel development server.
21. Open the GET route that displays the form.
22. Submit invalid data and inspect the error feedback.
23. Submit valid data and confirm the success redirect.
24. Refresh after success and confirm that the browser does not repeat the POST automatically in the usual Post/Redirect/Get flow.

15.2 Thunder Client testing

Thunder Client is useful for testing routes directly, but it does not behave exactly like a browser form with a Laravel session.

For a web POST route protected by CSRF:

25. First request the form page using GET.
26. Keep the session cookie and obtain the CSRF token from the page or session flow.
27. Send the POST request with the matching cookie and token.
28. If you send a POST request to a web route without a valid token, a 419 response is expected.

For a stateless API route, test the API authentication and validation response appropriate to that API. Do not disable CSRF globally merely to make a manual POST request work.

15.3 Example JSON validation response

When Laravel detects a JSON request (for example, an API client sends Accept: application/json), validation errors are normally returned with HTTP status 422.

```
{
    "message": "The email field is required.",
    "errors": {
        "email": [
            "The email field is required."
        ]
    }
}
```

The exact message and response fields depend on the submitted data and validation rules.

16. Common errors and troubleshooting

419 Page Expired	A web POST/PUT/PATCH/DELETE request has a missing, expired or mismatched CSRF token, or its session is not being preserved.	Add @csrf; preserve the session cookie; verify session configuration and domain; do not bypass CSRF as a quick fix.
The POST method is not supported for this route	The form method and route method do not match, or the action URL is wrong.	Check action, route URL and method. Use Route::post for POST forms.
The GET method is not supported	A browser or client is making GET while only a POST route exists.	Open the GET form route, then submit to the POST processing route.
old('name') is empty	No old input was flashed, the request did not redirect back, or the field name does not match.	Use validation redirect flow or withInput(); confirm the input name.
Errors are not displayed	The view is not receiving the error bag, or the wrong field key is used.	Use @error('field') and check the exact name attribute.
Undefined variable $request	The method does not receive the Request object or the wrong variable name is used.	Import Illuminate\Http\Request and type-hint Request $request.
Trying to access array offset on null / input access error	The code treats input() as an array property or assumes an optional value exists.	Use $request->input('name') and handle missing values.
Mass assignment exception	Model does not allow the fields being assigned.	Configure $fillable or $guarded, and persist only validated fields.
Validation passes but database insert fails	Validation rules do not cover database constraints or schema requirements.	Align validation, database constraints, casts and model configuration.

17. Security and best practices

- Use GET for reading/searching and POST/PUT/PATCH/DELETE for state-changing operations.
- Use @csrf in every normal Blade form that submits to a web route using the web middleware.
- Keep HTTPS enabled in production. POST alone does not encrypt information.
- Validate on the server. Client-side validation is only a convenience.
- Use validated data, not raw $request->all(), for database writes.
- Use authorization policies or gates to check whether the current user may perform the action.
- Use Blade's escaped output {{ }} when displaying user input.
- Never flash passwords, OTPs, API secrets or other sensitive values back into a form.

- Use password and password_confirmation fields with the confirmed rule; hash passwords before storage using Laravel's hashing facilities.
- Use database constraints as an additional integrity layer; validation does not replace constraints.
- Do not use GET routes for delete, payment, account changes or other state-changing actions.
- Use Form Request classes when validation logic becomes large or reusable.
- Give users clear, specific error messages without exposing internal stack traces or database details.

18. Classroom exercises and assessment questions

18.1 Practical exercises

1. Create a contact form with name, email, subject and message. Use GET to display the form and POST to process it.

2. Create a student admission form with name, email, phone, date of birth and course. Validate every field.

3. Add custom messages for required, email, digits and max rules.

4. After validation failure, preserve name, email and course using old().

5. Display all validation errors at the top and individual errors below each field.

6. Add a terms checkbox and validate it using accepted.

7. Create a profile update form that uses unique email validation while ignoring the current user's ID.

8. Create a document upload form that accepts only PDF files up to 2 MB.

9. Use flash data to display a success message after a successful submission.

10. Test a web POST route without @csrf and explain why a 419 response occurs.

18.2 Short-answer questions

1. What is the purpose of the Laravel Request object?

2. Differentiate between GET and POST.

3. What is the difference between input() and query()?

4. Why should a GET request not perform a delete operation?

5. What is CSRF? Explain a simple attack scenario.

6. What does the @csrf directive generate?

7. What is the usual meaning of HTTP 419 in a Laravel web form?

8. What is flash data? Give one example.

9. What does old('email') do?

10. What is the difference between $request->all() and $request->validated()?

11. What happens when validate() fails for a normal web request?

12. What happens when validation fails for a JSON request?

13. Explain required, nullable and sometimes.

14. What is the purpose of the confirmed validation rule?

15. How do you write a custom message for email.required?

16. What is the purpose of a Form Request class?

17. Why is server-side validation necessary even if HTML required is used?

18. Why should passwords not be flashed back as old input?

18.3 Suggested practical assessment rubric

Routes and HTTP methods	10
Correct Request handling	10
CSRF implementation	10
Validation rules and custom messages	25
Old input and error display	15
Flash success feedback and redirects	10
Code quality and security	10
Testing and explanation	10
Total	**100**

19. Quick revision sheet

Display a form	Route::get('/form', ...);
Process form	Route::post('/form', ...);
Request import	use Illuminate\Http\Request;
Read input	$request->input('name');
Read query string	$request->query('search');
All request input	$request->all();
Selected fields	$request->only(['name', 'email']);
CSRF in Blade	@csrf
Validate	$validated = $request->validate([...]);
Old input	old('name')
One field error	@error('name') {{ $message }} @enderror
All errors	$errors->all()
Flash message	->with('success', 'Saved!')
Manual validator	Validator::make($request->all(), [...])
Validated Form Request data	$request->validated()

Teaching summary

A secure Laravel form workflow follows a predictable pattern: show the form with GET, submit it to a state-changing route with POST, include a CSRF token, validate every field on the server, process only validated data, and redirect with useful feedback. When validation fails, Laravel provides an error bag and flashes eligible old input so the user can correct the form without starting again.

Core pattern to remember GET form → POST submit → CSRF check → validate → save/process validated data → redirect with flash message. If validation fails: redirect back → show errors → restore old input.